ChatGPT said:

QC & Project Audit Agent — Production Implementation Blueprint

Architecture defined

Approval-based remediation

My recommendation is to build this as a **reusable Autonomous Software Quality Intelligence Platform**, not simply a code-review application.

The product will have two operating experiences:

1. **Standalone QC Platform:** Open a dashboard, select any software repository, and initiate a complete technical audit.
2. **Project-Integrated QC Agent:** Install a lightweight agent/CLI inside an existing project. It can audit that project on demand, after code changes, or before releases.

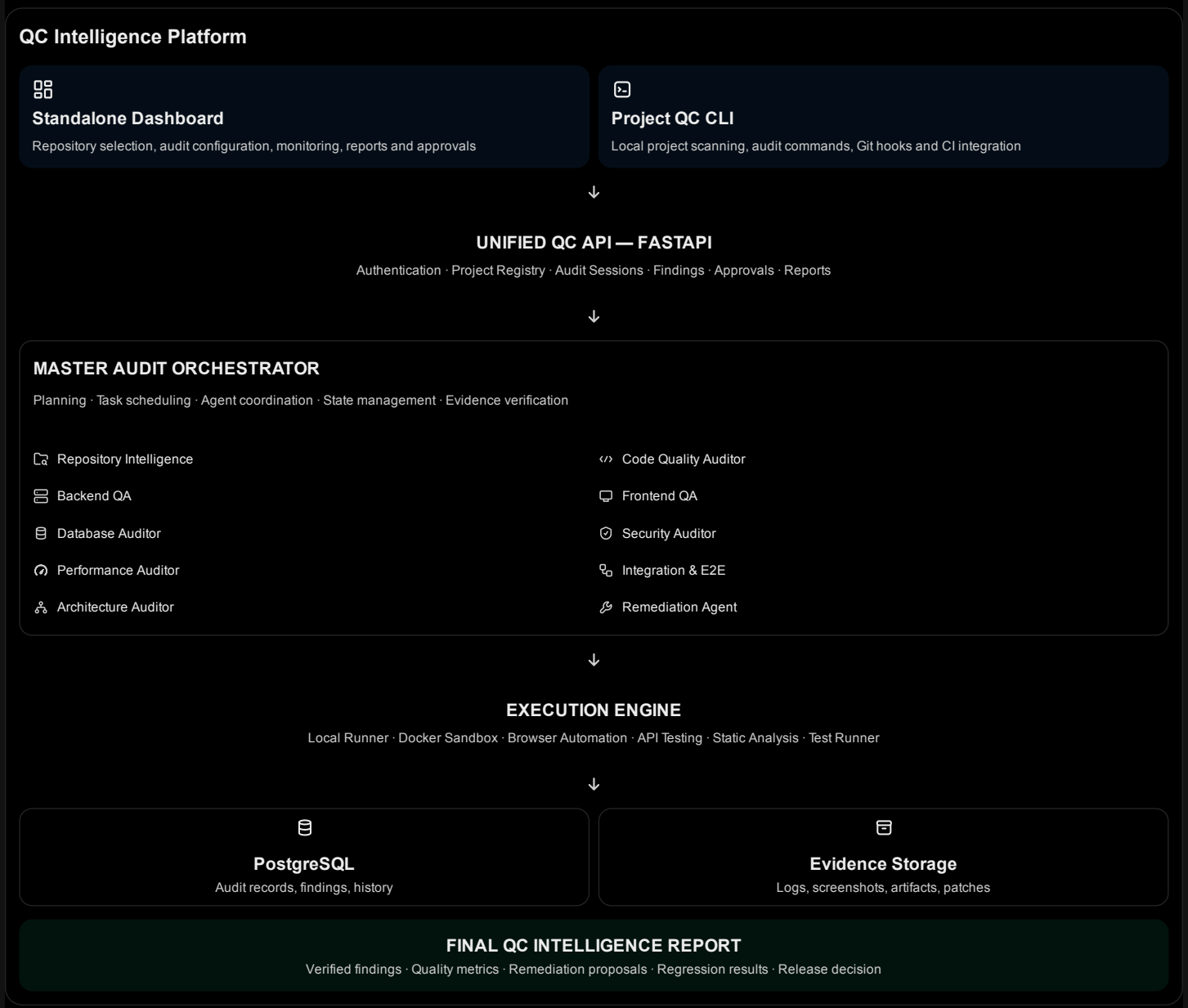
Both experiences must use the same audit engine, execution infrastructure, finding database, and reporting system.

The most important architectural decision is to separate AI reasoning from actual execution. AI decides what to inspect, interprets evidence, identifies root causes, and proposes fixes. Deterministic tools execute commands, tests, browser interactions, and security checks.

This prevents the system from claiming that something works merely because an LLM thinks the code looks correct.

---

1. Product architecture



2. Recommended technology stack

Use technologies that make the platform maintainable and give you direct control over the execution pipeline.

Layer	Recommended technology
Backend	Python 3.11+, FastAPI
Frontend	React, Vite, TypeScript
UI	Tailwind CSS
Database	PostgreSQL
ORM	SQLAlchemy 2.x
Validation	Pydantic V2
Browser automation	Playwright
Backend testing	Pytest, HTTPX
Static analysis	Ruff, ESLint, Mypy, Semgrep
Security	Bandit, OSV Scanner, Semgrep
Performance	k6, Locust
Container execution	Docker
Background jobs	Redis + Celery or a controlled worker queue
AI providers	Gemini, OpenAI, Ollama
Logging	Structured JSON logging
Report generation	HTML + PDF

AI architecture recommendation: Build your own lightweight AgentService and ToolRegistry rather than making LangChain or LangGraph the foundation.

This gives you control over tool execution, prompts, error handling, state transitions, and audit evidence.

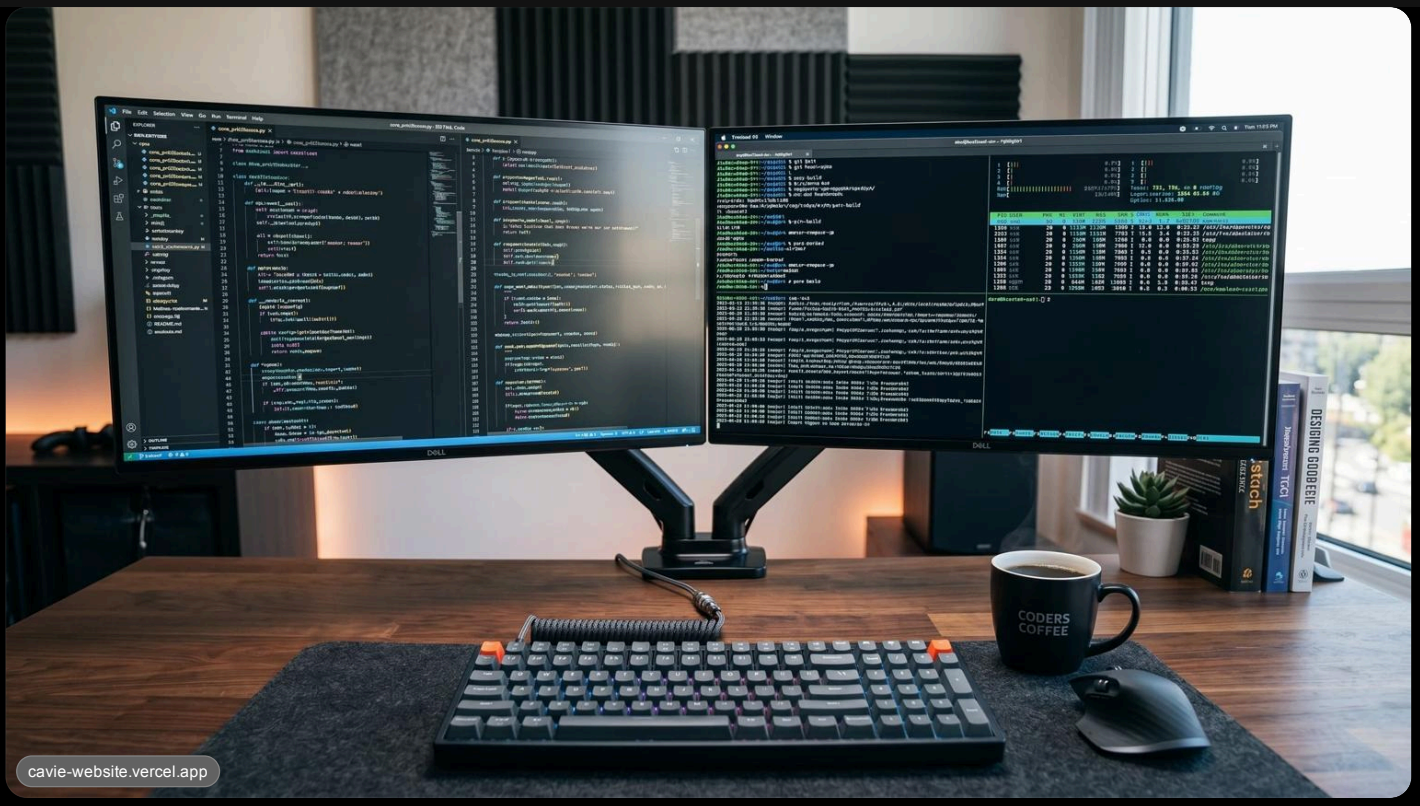
Use a provider-independent LLM gateway so switching between Gemini, OpenAI and local Ollama models does not require changing the audit engine.

---

3. Two execution modes

This is a major part of the architecture.

Mode A — Local execution Developer mode



The platform communicates with a local QC runner that has access to explicitly authorized project directories.

Workflow:

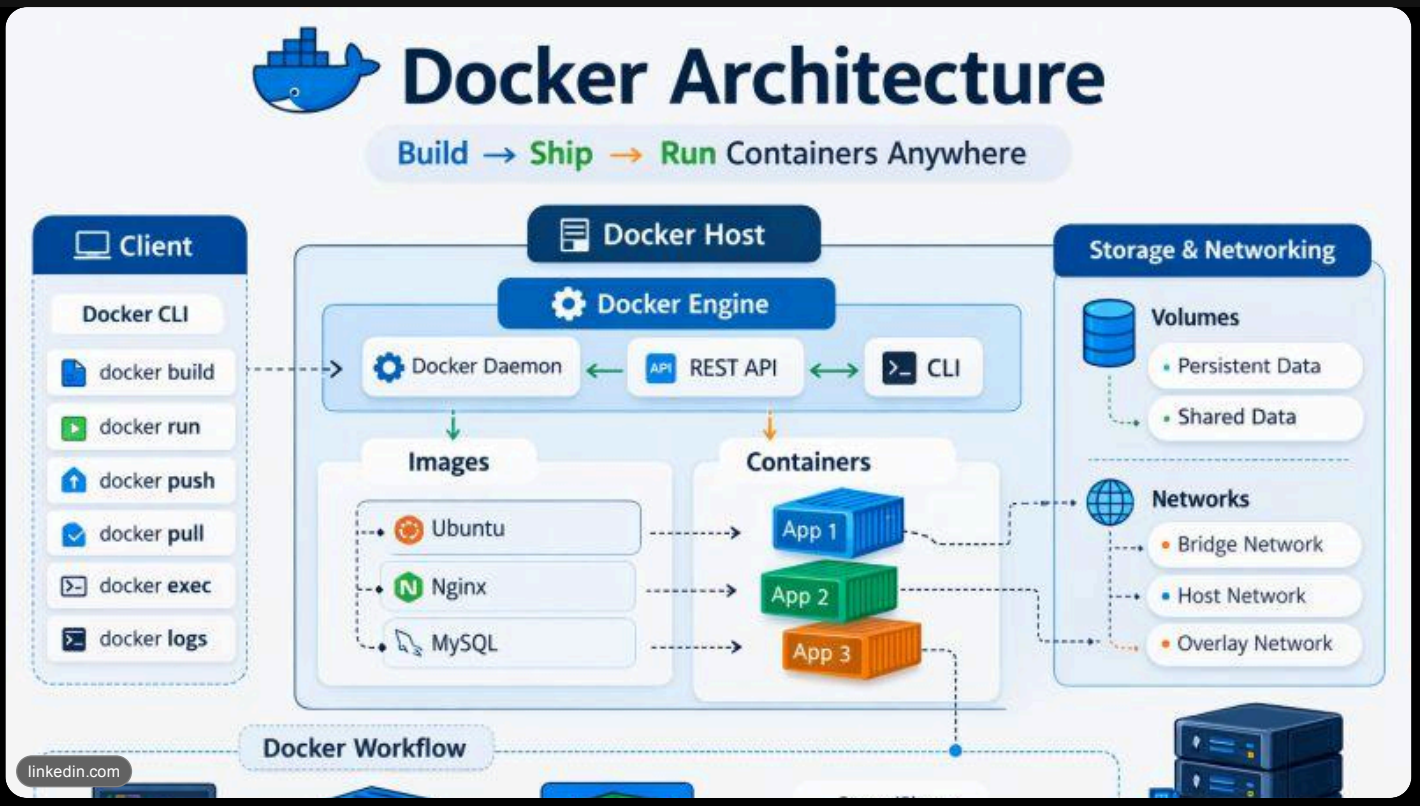
1. User selects a project directory.
2. Runner inventories repository files.
3. Runner analyzes the source.
4. Runner creates an isolated audit workspace.
5. Backend and frontend are started with controlled processes.
6. Browser automation executes tests.
7. Evidence is returned to the central audit engine.

Advantages:

- Easy access to existing repositories.
- No requirement to upload large projects.
- Useful during active development.

Limitation: local execution is less isolated than a properly configured container. Never run untrusted code with unrestricted host permissions.

Mode B — Docker execution Recommended for untrusted code



The platform copies or mounts an authorized project snapshot into a disposable sandbox.

Workflow:

1. Create an isolated workspace.
2. Build or select a project-specific environment.
3. Install dependencies using controlled package access.
4. Start services inside the sandbox.
5. Execute test suites.
6. Collect logs, screenshots and results.
7. Destroy the execution environment.

Advantages:

- Reproducible test environments.
- Better isolation.
- Easier cleanup.
- Resource and execution limits.

Docker itself is not a complete security boundary. Apply non-root execution, restricted capabilities, resource limits, network policies, read-only mounts where practical, and never mount the host Docker socket into the audited project.

Important: Both modes should use the same Runner interface. The audit engine should not care whether a command runs locally or inside Docker.

4. Agent orchestration design

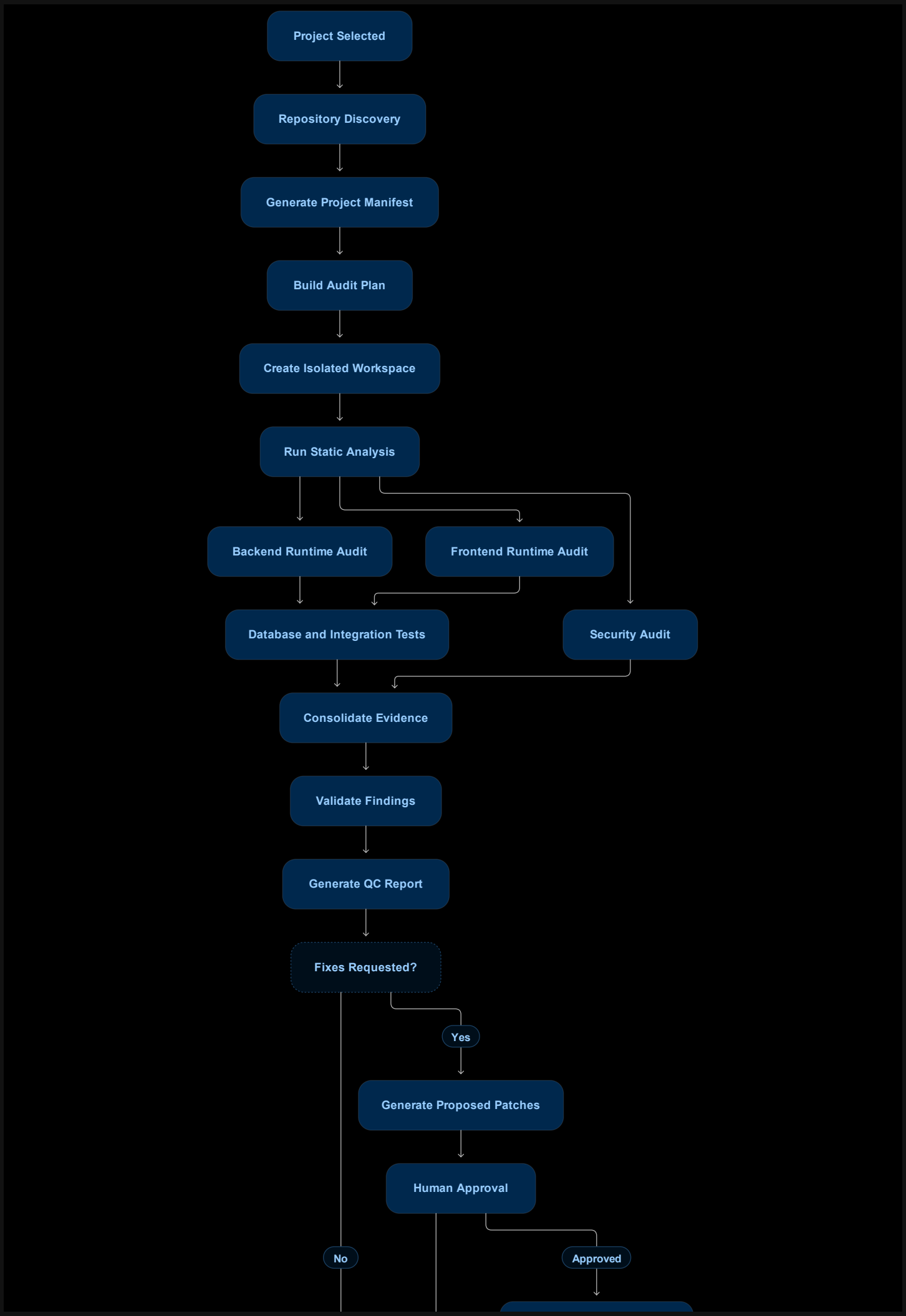
Do not let ten independent agents freely communicate and execute commands. That creates unpredictable behavior, duplicated work, and expensive LLM calls.

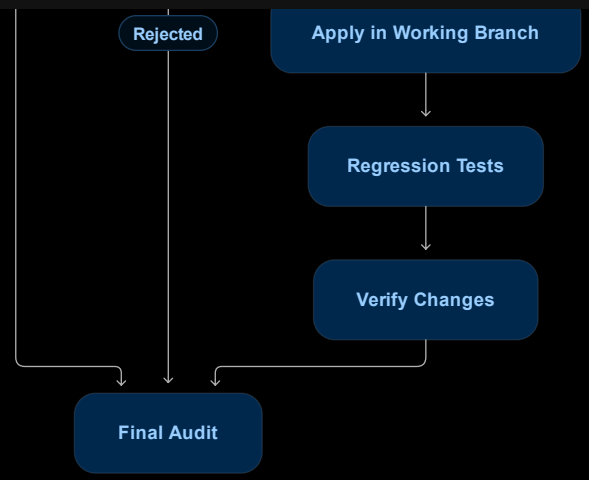
Use a central orchestrator with specialized agents that receive narrowly defined tasks.

Agent	Responsibility	Main tools
Repository Intelligence	Discover project structure and stack	File scanner, Git
Code Auditor	Inspect code and logic	AST, Ruff, ESLint
Architecture Auditor	Identify design and dependency issues	Dependency graph, static analysis
Backend QA	Start backend and test APIs	Pytest, HTTPX
Frontend QA	Inspect browser and UI	Playwright

Database Auditor	Inspect schema and data integrity	SQLAlchemy, DB inspection
Security Auditor	Detect vulnerabilities	Semgrep, Bandit, OSV
Performance Auditor	Measure runtime performance	k6, profiling
Integration Auditor	Validate complete workflows	Playwright, HTTPX
Remediation Agent	Propose changes and patches	Git diff, patch engine

Orchestration workflow


Project Selected
Repository Discovery
Generate Project Manifest
Build Audit Plan
Create Isolated Workspace
Run Static Analysis
Backend Runtime Audit
Frontend Runtime Audit
Database and Integration Tests
Security Audit
Consolidate Evidence
Validate Findings
Generate QC Report
Fixes Requested?
Yes
Generate Proposed Patches
Human Approval
No
Approved



Agent execution rules

Each agent must return a standardized result rather than free-form text.

For example:

```python
class AuditFinding(BaseModel):
    finding_id: str
    category: str
    severity: str
    title: str
    description: str

    file_path: str | None
    line_start: int | None
    line_end: int | None

    evidence_ids: list[str]
    reproduction_steps: list[str]

    root_cause: str | None
    recommended_fix: str | None

    confidence: float
    verification_status: str
```

A finding without evidence should not be marked as a confirmed defect.

The orchestrator should also track which files and test targets were inspected. This is essential for proving audit coverage.

5. Production database architecture

Use PostgreSQL as the central source of truth.

The database must support multiple projects, multiple audit sessions, audit history, findings, approvals, execution logs, and remediation tracking.

Core database schema

Database entity relationships

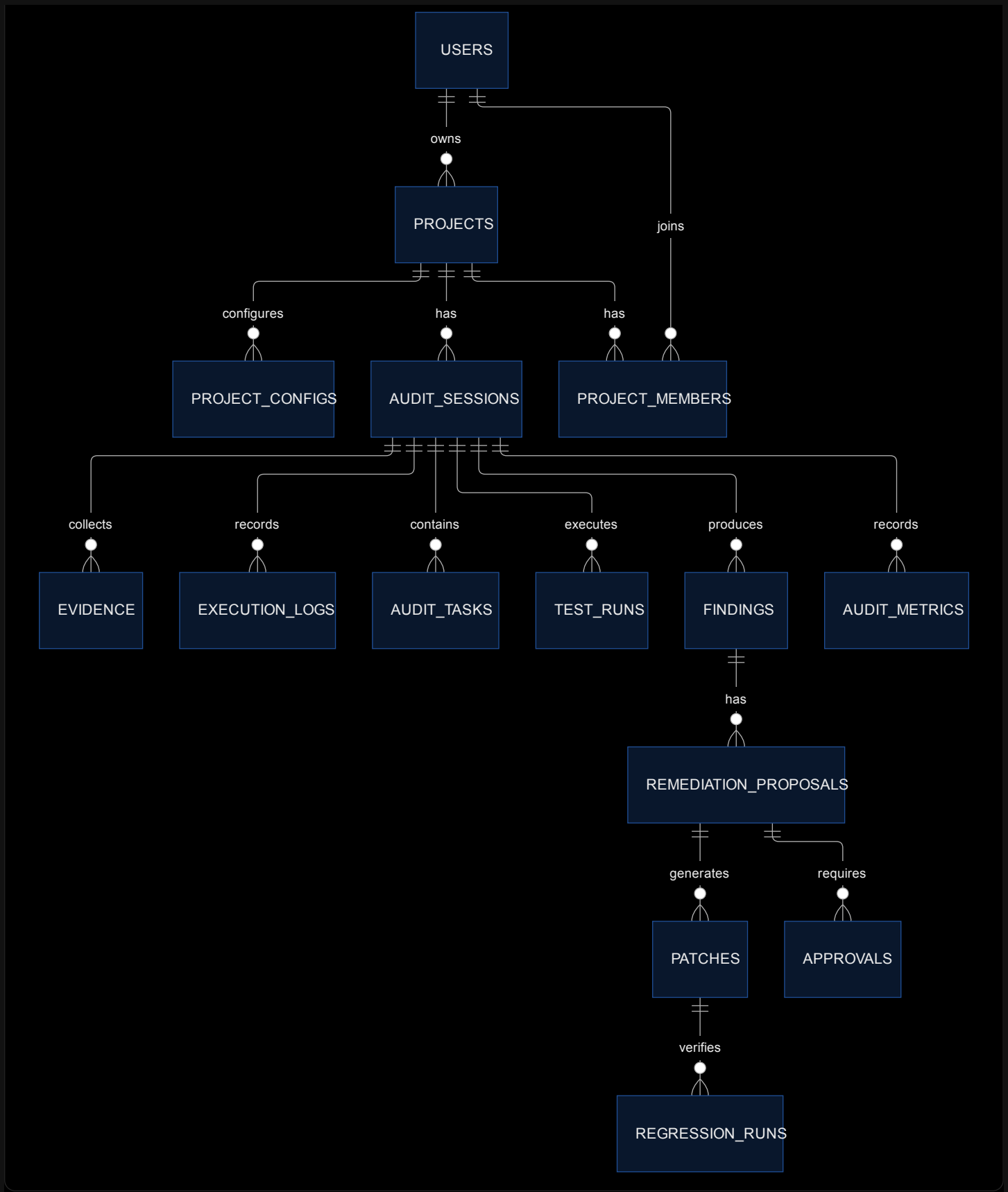


Table definitions

Table	Important fields	Purpose
users	id, email, password_hash, role	User accounts
projects	id, name, root_path, repository_url, stack	Project registry
project_members	project_id, user_id, permissions	Access control
project_configs	project_id, config_json, exclusions	Audit configuration
audit_sessions	id, project_id, status, mode, started_at, completed_at	Audit history

audit_tasks	id, session_id, agent_type, status, attempts	Agent execution
findings	id, session_id, severity, category, fingerprint, status	Identified issues
evidence	id, session_id, finding_id, artifact_uri, checksum	Supporting proof
test_runs	id, session_id, tool, command, status, duration	Test execution
audit_metrics	id, session_id, metric_name, value, metadata	Quality metrics
execution_logs	id, session_id, task_id, command, exit_code, output_uri	Execution history
remediation_proposals	id, finding_id, patch_id, status	Proposed fixes
approvals	id, proposal_id, reviewer_id, decision, timestamp	Human decisions
patches	id, proposal_id, diff_uri, checksum, applied_at	Code changes
regression_runs	id, patch_id, test_run_id, result	Fix validation

Additional production requirements

- UUID primary keys.
- UTC timestamps.
- Foreign-key constraints.
- Database migrations using Alembic.
- Indexes on `project_id`, `session_id`, `severity`, `status`, and `fingerprint`.
- JSONB for flexible audit configuration and structured metadata.
- Retention policies for logs and large artifacts.
- Soft deletion for user-facing project records where appropriate.

Store screenshots, videos, build artifacts, and full logs in object storage or controlled filesystem storage. Do not put large binary files directly into PostgreSQL.

Finding lifecycle

Every issue should have an explicit lifecycle:

1. **Detected**

   A scanner or agent identifies a possible issue.
2. **Evidence collected**

   Logs, code locations, screenshots, or test results are attached.
3. **Validated**

   A suitable independent check confirms or rejects the finding.
4. **Remediation proposed**

   A patch and explanation are generated.
5. **Awaiting approval**

   Human reviewer examines the proposed change.
6. **Applied**

   Approved patch is applied to a dedicated working branch or copy.
7. **Regression verified**

   Tests establish whether the change resolves the issue without introducing regressions.
8. **Resolved / Reopened**

   The finding is closed only when verification criteria are satisfied.

One important distinction: a proposed patch being approved does not mean the finding is resolved. It is resolved only after verification.

6. Complete audit engine design

This is the core intelligence of the product.

A. Repository Intelligence Engine

Its job is to build a complete picture of the project before any audit begins.

It should generate a machine-readable manifest:

JSON

```json
{
  "project_name": "sample-project",
  "languages": ["Python", "TypeScript"],
  "frameworks": ["FastAPI", "React"],
  "backend": {
    "detected": true,
    "entrypoint": "backend/main.py"
  },
  "frontend": {
    "detected": true,
    "entrypoint": "frontend/package.json"
  },
  "database": ["PostgreSQL"],
  "test_frameworks": ["pytest", "playwright"],
  "total_files": 245,
  "inspected_files": 0,
  "excluded_files": [],
  "discovery_complete": true
}
```

The actual manifest should include:

- File paths
- File types
- File sizes
- Content hashes
- Language classification
- Inspection status
- Exclusion reasons
- Dependency relationships
- Detected entrypoints

Ignore generated directories such as `node_modules`, `.venv`, build outputs, and Git internals by default, but explicitly record exclusions.

Do not silently ignore source files because they are large or difficult to parse. Chunk large files and track incomplete analysis.

B. Static Code Audit Engine

Use deterministic analysis first:

1. AST-based analysis.
2. Linting.
3. Type checking.
4. Dependency graph analysis.
5. Security scanning.
6. Complexity analysis.
7. Duplicate code detection.

Then use the LLM to interpret the findings and discover contextual logic problems.

For example, a linter can detect an unused function, but an LLM can investigate whether the function represents an incomplete business workflow.

C. Runtime Execution Engine

Create a universal execution interface.

Python

```python
class ProjectRunner:
    async def prepare(self, project, config):
        ...
```

```
    async def execute(self, command, timeout):
        ...

    async def start_service(self, service_config):
        ...

    async def stop_service(self, service_id):
        ...

    async def collect_logs(self, service_id):
        ...

    async def cleanup(self):
        ...
```

Implement two adapters:

- `LocalProjectRunner`
- `DockerProjectRunner`

The orchestrator communicates only with the interface, never directly with Docker or local process management.

All commands must be associated with an audit task, execution timeout, working directory, resource policy, and captured result.

D. Frontend testing engine

Use Playwright to inspect the actual rendered application.

Test:

- Page loading.
- Route navigation.
- Forms and validation.
- Buttons and actions.
- API connectivity.
- Browser console errors.
- Failed network requests.
- Responsive layouts.
- Accessibility basics.
- User workflows.

The agent should discover pages from router configurations, navigation links, and runtime crawling.

It should create screenshots and test traces for reproducible failures.

E. Backend testing engine

Discover endpoints through:

- OpenAPI specifications.
- Framework route inspection.
- Existing test suites.
- Source-code route discovery.

Test endpoint behavior with valid, invalid, missing, and boundary-case inputs.

Never indiscriminately call DELETE or other destructive endpoints against real project data. Use isolated test data and a controlled database.

F. Database validation engine

Check schema definitions, migrations, relationships, constraints, and actual persistence.

For every CRUD workflow, verify both the API response and database state.

G. Security engine

Use scanners to identify concrete issues.

AI should help assess contextual impact and false positives, not invent vulnerabilities.

Use an authorization boundary for every project and execution environment.

H. Performance engine

Run measurable tests and report:

- Test environment.
- Dataset size.
- Concurrent users.
- Request volume.
- Latency percentiles.
- Error rates.
- CPU and memory consumption.
- Bottleneck evidence.

A performance report without test conditions is not a valid performance report.

---

7. QC scoring and release gate

Do not use one arbitrary AI-generated percentage to determine whether a project is good.

Use a transparent scorecard.

Dimension	Suggested weight
Code quality	15%
Backend reliability	15%
Frontend quality	10%
Architecture	10%
Security	20%
Integration and functionality	15%
Performance	10%
Test quality	5%
Total	100%

The weighted score is a reporting aid, not a substitute for release gates.

Illustrative release quality gate

90–100	Strong
75–89	Acceptable with review
60–74	Needs improvement
Below 60	Not recommended

Independent blocking conditions

- Unresolved confirmed critical vulnerabilities.
- Critical business workflow failures.
- Unresolved critical data integrity defects.
- Essential audit phases not completed.
- Unverified changes in the release candidate.

A project can score 95 and still be rejected if it contains a confirmed critical vulnerability.

Likewise, an incomplete audit must never receive an approval simply because no issues were found.

8. Project-integrated agent design

The standalone platform and integrated agent must share the same core engine.

I recommend developing a lightweight Python CLI package called:

`qc-agent`

Install it in any supported software repository.

Example project layout:

Plain text

```
my-project/
│
├── .qc/
│   ├── config.yaml
│   ├── ignore
│   └── audit-rules.yaml
│
├── qc-reports/
│
├── backend/
├── frontend/
├── tests/
│
├── pyproject.toml
└── README.md
```

Example commands:

Bash

```
# Inspect project
qc-agent discover

# Run a complete audit
qc-agent audit --full

# Run code-quality checks
qc-agent audit --code

# Run backend checks
qc-agent audit --backend

# Run frontend checks
qc-agent audit --frontend

# Run security checks
qc-agent audit --security

# Run end-to-end checks
qc-agent audit --e2e

# Run using Docker
qc-agent audit --full --runner docker

# View findings
qc-agent findings

# Generate report
qc-agent report --format html
```

The CLI can operate independently or connect to the central QC platform.

Integration methods

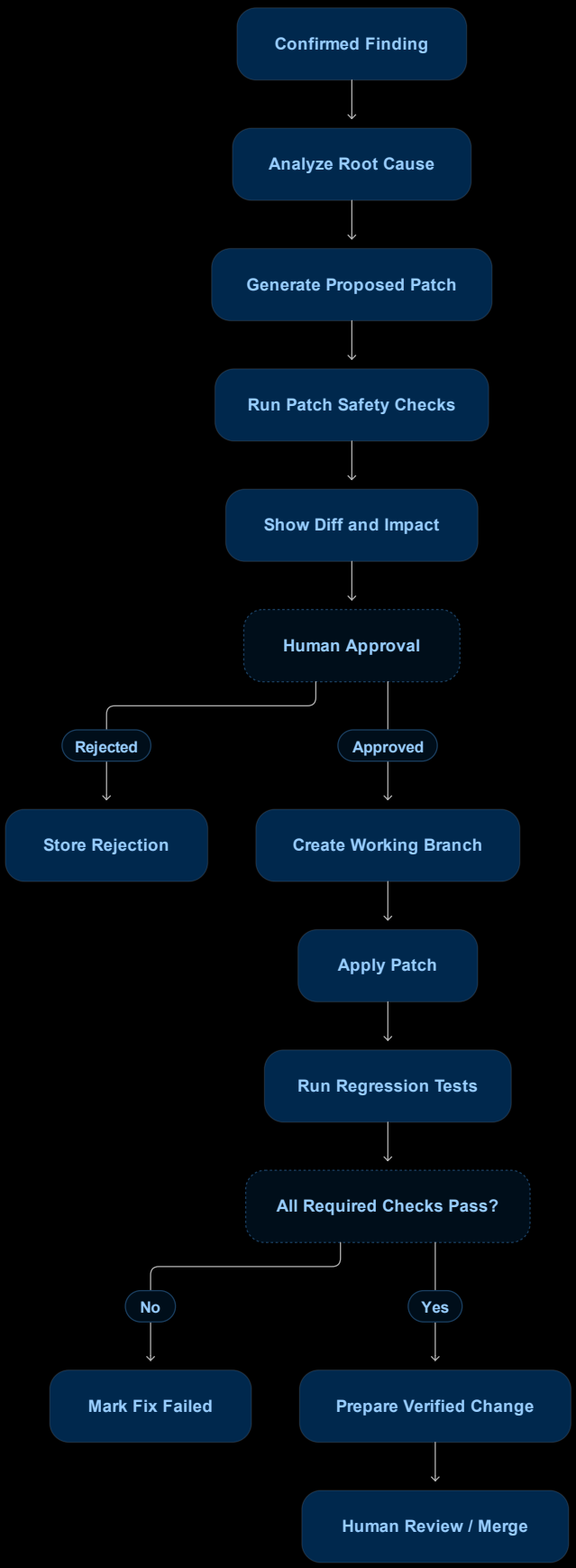
Integration	Purpose
Manual CLI	Run audits on demand
Git pre-commit	Catch basic quality problems before committing

GitHub Actions	Run checks on pushes and pull requests
CI/CD quality gate	Block releases when defined conditions fail
Scheduled audit	Periodic comprehensive audit
Standalone dashboard	Centralized audit management

Do not run a full browser-based audit on every keystroke or Git commit. Use incremental checks for small changes and complete audits for release candidates or scheduled reviews.

9. Approval-based remediation architecture

This needs to be especially strict.



Mandatory approval controls

Before approval, show:

- Original code.
- Proposed code.
- Full Git diff.
- Files affected.
- Finding being addressed.
- Expected behavior.

- Potential side effects.
- Tests that will be run.
- Rollback procedure.

Additional rules:

1. The original source must remain recoverable.
2. Patches must be applied to a dedicated branch or working copy.
3. The agent cannot merge into the main branch automatically.
4. No production deployment.
5. A failed regression test must mark the patch as failed.
6. Approval must be tied to a particular patch checksum.
7. Any modification to an approved patch invalidates that approval.

This prevents an agent from receiving approval for one patch and then applying a different one.

10. Production security architecture

This product executes code supplied by users. That is its biggest security risk.

A malicious repository can contain scripts that attempt to steal credentials, access the host filesystem, or communicate with external services.

Implement the following:

Security layer	Requirement
Workspace isolation	Dedicated workspace per audit
Container	Non-root user
Filesystem	Restricted mounts and read-only source where possible
Network	Disabled by default; controlled package and test access
Resources	CPU, memory, disk and process limits
Execution	Command allowlists and timeouts
Credentials	No host environment secrets passed to projects
Database	Isolated disposable test database
Approval	Human authorization for mutations
Cleanup	Destroy containers and temporary workspaces
Audit trail	Record every executed command

For local execution, explicitly display the reduced isolation warning and restrict commands to an approved execution policy.

The system should never expose unrestricted terminal access to an LLM.

The LLM selects from registered tools. The execution layer validates every request before running it.

11. Implementation roadmap — 10 phases

I recommend implementing this in vertical slices, not generating the entire application in one enormous coding session.

Each phase must produce executable functionality.

Development roadmap

1 **Phase 0 — Foundation and architecture**

First milestone

- Repository structure.
- FastAPI foundation.
- React dashboard shell.
- PostgreSQL and Alembic.

- Authentication and project access.
- Structured logging and configuration.
- Audit state machine.

Deliverable: Running platform with project registration and persistent audit sessions.

2 **Phase 1 — Repository Intelligence**

- Local project selection.
- Project upload or Git repository import.
- Recursive file inventory.
- Technology detection.
- Dependency discovery.
- File hashing and exclusions.
- Inspection coverage tracking.

Deliverable: Accurate project manifest and repository explorer.

3 **Phase 2 — Static Code Audit**

- Language-specific analyzers.
- Linting and type checking.
- Code complexity.
- Dependency graph.
- Duplicate detection.
- AI-assisted code reasoning.
- Finding deduplication.

Deliverable: Evidence-backed source code audit.

4 **Phase 3 — Execution Infrastructure**

- Local runner.
- Docker runner.
- Environment preparation.
- Process lifecycle.
- Command execution.
- Timeout handling.
- Log and artifact collection.

Deliverable: Controlled execution of real projects.

5 **Phase 4 — Backend QA**

- Server startup.
- Endpoint discovery.
- API tests.
- Request and response validation.
- Database connectivity.
- CRUD verification.
- Runtime diagnostics.

Deliverable: Working backend quality auditor.

6 **Phase 5 — Frontend QA**

- Browser automation.
- Route discovery.
- UI interaction testing.
- Console and network inspection.
- Responsive checks.
- Screenshot evidence.

Deliverable: Working browser-based frontend auditor.

7 **Phase 6 — Integration, Security and Performance**

- End-to-end workflows.
- Data consistency checks.
- Security scanning.
- Dependency vulnerabilities.
- Performance benchmarking.

- Cross-component validation.

Deliverable: Full technical quality assessment.

8 **Phase 7 — AI Orchestration**

- Master orchestrator.
- Specialized agent registry.
- Tool selection.
- Task scheduling.
- Evidence validation.
- LLM provider abstraction.
- Interrupted audit recovery.

Deliverable: Coordinated autonomous audit system.

9 **Phase 8 — Remediation and Approval**

- Root-cause analysis.
- Patch generation.
- Git diff viewer.
- Approval workflows.
- Patch application.
- Regression testing.

Deliverable: Controlled AI-assisted repair system.

10 **Phase 9 — Platform Integration and Enterprise Hardening**

- QC CLI.
- GitHub Actions integration.
- CI/CD quality gates.
- HTML/PDF reporting.
- Multi-project support.
- RBAC and audit logs.
- Backup, monitoring and deployment.

Deliverable: Production-ready reusable QC platform.

12. Acceptance criteria for the finished product

These are the conditions I would use to determine whether the implementation is genuinely complete.

Capability	Acceptance condition
Repository scanning	Every in-scope file accounted for
Technology detection	Correctly identifies mixed-stack projects
Code auditing	Produces file-level findings with evidence
Backend testing	Actually starts and tests the backend
Frontend testing	Actually opens and interacts with the browser
Database validation	Confirms actual persistence
E2E testing	Executes complete user workflows
Security	Runs deterministic security scanners
Performance	Produces measured benchmark results
Findings	Deduplicated and traceable
Remediation	Produces reviewable patches
Approval	Prevents unauthorized modifications
Regression	Re-tests approved fixes
Reports	Clearly distinguishes completed and incomplete checks
CLI	Works independently of the dashboard
Docker	Executes projects in disposable sandboxes

Local	Executes explicitly authorized project directories

Mandatory dogfooding test

Before declaring the product complete, audit the QC platform itself.

It must:

1. Discover its own frontend and backend.
2. Inventory its own source files.
3. Run its backend.
4. Run its frontend.
5. Execute its API tests.
6. Execute browser workflows.
7. Validate its database operations.
8. Run security checks.
9. Generate its own audit report.
10. Produce and verify at least one controlled remediation workflow using a deliberately introduced test defect.

That last test is important: it demonstrates that the platform can find a real defect, propose a patch, wait for approval, and verify the fix.

---

13. Refined master development prompt

The following is the main implementation prompt. Use it in Antigravity or another autonomous coding IDE.

It is intentionally more prescriptive than a normal project-generation prompt. It defines the architecture, operating rules, implementation sequence, security requirements, and acceptance criteria.

MASTER BUILD PROMPT

AUTONOMOUS QC & PROJECT AUDIT INTELLIGENCE PLATFORM

Version 1.0 — Production Architecture and Implementation Specification

---

1. YOUR ROLE

Act as a Principal Software Architect, Senior Python Engineer, Full-Stack Developer, Autonomous Agent Architect, QA Automation Engineer, Security Engineer, DevOps Engineer, and Technical Project Manager.

You are responsible for architecting and building a production-grade Autonomous QC & Project Audit Intelligence Platform.

You must work as an autonomous engineering team.

Do not build a static dashboard, a simulated agent, a code-review chatbot, or a demonstration-only application.

Build a genuinely operational platform capable of inspecting, executing, testing, validating, diagnosing, and auditing complete software projects.

The product must have two operating modes:

MODE A — STANDALONE QC PLATFORM

A centralized dashboard where users can register, upload, import, or select software projects and execute comprehensive technical audits.

MODE B — PROJECT-INTEGRATED QC AGENT

A lightweight Python CLI and integration package that can be installed in any supported software repository and execute audits against that project.

Both modes must use the same core audit engine, finding schemas, execution abstractions, evidence collection, and reporting system.

The product must support both local execution and isolated Docker execution.

All code modifications must require explicit human approval.

---

2. PRODUCT VISION

The system must function as an independent software quality-control department.

Whenever a project is under development, the agent should be capable of:

1. Understanding the entire project architecture.
2. Inspecting every in-scope source file.
3. Understanding frontend and backend relationships.
4. Identifying the database architecture.
5. Identifying unfinished functionality.
6. Running the actual backend.
7. Running the actual frontend.
8. Testing APIs.
9. Interacting with the actual browser application.
10. Testing complete business workflows.
11. Validating database persistence.
12. Detecting code defects.
13. Identifying architectural weaknesses.
14. Detecting security vulnerabilities.
15. Measuring performance.
16. Identifying broken integrations.
17. Generating evidence-backed findings.
18. Proposing code fixes.
19. Waiting for human approval.
20. Applying approved changes in an isolated working branch.
21. Running regression tests.
22. Generating a complete technical audit report.

The platform must never confuse code inspection with actual functional verification.

A feature is not considered operational simply because a corresponding function, component, or API route exists.

---

3. NON-NEGOTIABLE ENGINEERING PRINCIPLES

Follow these principles throughout development.

3.1 Evidence First

Every confirmed finding must have supporting evidence.

Evidence may include:

- Source code locations.
- Test failures.
- Stack traces.
- Browser screenshots.
- Browser traces.
- HTTP responses.
- Database inspection results.
- Security scanner output.
- Performance measurements.
- Execution logs.

Never fabricate findings or test results.

3.2 Complete Inspection Tracking

Maintain a repository manifest.

Every discovered file must have an explicit inspection status:

- Pending
- Inspected
- Excluded
- Unsupported
- Failed
- Partially inspected

Every excluded file must have a reason.

Do not silently skip large files, unfamiliar file types, or complex modules.

The platform must distinguish:

- Files discovered.
- Files in scope.
- Files inspected.
- Files excluded.
- Files not inspected.
- Files with failed analysis.

3.3 Deterministic Execution

AI must not determine whether a test passed based on reasoning.

Tests pass or fail based on actual execution results.

AI is responsible for analysis, planning, root-cause investigation, and interpretation.

3.4 Human-Controlled Remediation

The agent can independently identify issues and prepare fixes.

It cannot modify original project files without approval.

3.5 Modularity

Use clean architecture, SOLID principles, dependency injection, repository patterns, strict typing, and well-defined service boundaries.

3.6 Reproducibility

Every audit must record:

- Project snapshot or Git commit.
- File hashes.
- Environment details.
- Tool versions.
- Commands executed.
- Execution results.
- Audit configuration.
- AI provider and model.
- Audit timestamps.

A completed audit must be reproducible against the same project snapshot and environment to the extent the external dependencies allow.

---

4. TECHNOLOGY STACK

Use the following stack unless an actual technical incompatibility is identified.

Backend

- Python 3.11+
- FastAPI
- Pydantic V2
- SQLAlchemy 2.x
- Alembic
- PostgreSQL
- Redis
- Background worker system

Frontend

- React
- TypeScript
- Vite

- Tailwind CSS
- Recharts
- React Router

Testing and Execution

- Pytest
- HTTPX
- Playwright
- Ruff
- Mypy
- ESLint
- Semgrep
- Bandit
- OSV Scanner
- k6
- Docker
- Docker Compose

AI

Implement a provider-independent LLM gateway.

Support:

- Google Gemini
- OpenAI
- Ollama

Use local Ollama execution where practical to reduce API costs.

Use cloud providers when local models are insufficient for a particular analysis.

All prompts must be maintained in external prompt templates. Do not hardcode large prompts inside business logic.

Infrastructure

- PostgreSQL for persistent application data.
- Redis for queues, distributed coordination, and temporary state where required.
- Filesystem or S3-compatible object storage for evidence and artifacts.
- Docker for isolated execution.
- Structured JSON logging.

Do not introduce unnecessary dependencies.

---

5. REQUIRED PROJECT STRUCTURE

Use a modular monorepo.

qc-intelligence-platform/

```
backend/
    app/
        api/
            v1/
                endpoints/
            dependencies/

        core/
            config.py
            security.py
            logging.py
            exceptions.py

        domain/
```

```
        entities/
        enums/
        interfaces/
        value_objects/

    application/
        services/
        use_cases/
        dto/

    infrastructure/
        database/
        repositories/
        storage/
        queue/
        ai/
        execution/
        scanners/

    agents/
        orchestrator/
        registry/
        repository_intelligence/
        code_auditor/
        architecture_auditor/
        backend_qa/
        frontend_qa/
        database_auditor/
        security_auditor/
        performance_auditor/
        integration_auditor/
        remediation/

    audit_engine/
        discovery/
        planning/
        scheduling/
        execution/
        evidence/
        validation/
        scoring/
        reporting/

    tools/
        filesystem/
        git/
        terminal/
        docker/
        browser/
        api/
        database/
        security/
        performance/

    models/
    schemas/
    prompts/
    workers/
    middleware/
```

```
    tests/
        unit/
        integration/
        e2e/

    alembic/

frontend/
    src/
        app/
        components/
        layouts/
        pages/
        features/
            projects/
            audits/
            findings/
            evidence/
            remediation/
            approvals/
            reports/
            settings/
        hooks/
        services/
        stores/
        types/

qc_agent_cli/
    qc_agent/
        commands/
        discovery/
        configuration/
        client/
        local_execution/
        reporting/

    tests/

execution/
    docker/
    policies/
    templates/
    scripts/

shared/
    contracts/
    schemas/
    constants/

infrastructure/
    docker-compose.yml
    monitoring/
    deployment/

docs/
    architecture/
    api/
    security/
    operations/
    development/
```

```
scripts/
.env.example
README.md
```

Maintain strict separation between domain logic, orchestration, infrastructure, and API layers.

---

6. DATABASE IMPLEMENTATION

Implement PostgreSQL using SQLAlchemy 2.x and Alembic.

Create the following entities:

1. User
2. Project
3. ProjectMember
4. ProjectConfiguration
5. AuditSession
6. AuditTask
7. RepositoryFile
8. Finding
9. Evidence
10. TestRun
11. AuditMetric
12. ExecutionLog
13. RemediationProposal
14. Approval
15. Patch
16. RegressionRun
17. AgentExecution
18. Report
19. ToolExecution
20. AuditEvent

Required relationships

- One project can have multiple audit sessions.
- One audit session can have multiple audit tasks.
- One audit session can have many repository file records.
- One audit session can produce many findings.
- Every finding can have multiple evidence records.
- Every audit session can have multiple test runs.
- A finding can have multiple remediation proposals.
- A proposal can have approval records.
- A proposal can generate one or more patches.
- A patch can have multiple regression runs.
- Every agent execution belongs to an audit session and task.

Required database behavior

- UUID primary keys.
- UTC timestamps.
- Foreign-key constraints.
- Proper indexes.
- Unique finding fingerprints per relevant audit scope.
- Transaction management.
- Optimistic concurrency for approvals and patch state transitions where appropriate.
- Alembic migrations.
- Pagination for large findings and logs.
- Soft deletion where appropriate.
- Data retention configuration.

Use JSONB only for genuinely variable metadata.

Do not put the complete application state into one JSON column.

Store large artifacts outside PostgreSQL and persist their metadata, checksum, storage location, and retention information.

---

7. REPOSITORY INTELLIGENCE ENGINE

Build a technology-independent repository discovery engine.

It must support mixed-stack repositories.

Detect:

Languages

Python, JavaScript, TypeScript, Java, Go, C#, PHP, Ruby, Rust, HTML, CSS, SQL, Shell, YAML, JSON, and other identifiable languages.

Frameworks

FastAPI, Flask, Django, React, Next.js, Vue, Angular, Express, NestJS, Spring Boot, .NET, and other detected frameworks.

Databases

PostgreSQL, MySQL, SQLite, MongoDB, Redis, Firebase, Supabase, and others.

Infrastructure

Docker, Docker Compose, GitHub Actions, Kubernetes, and common deployment configurations.

Discovery process

1. Enumerate repository files.
2. Apply configurable exclusion policies.
3. Identify file types.
4. Identify package manifests.
5. Detect entrypoints.
6. Identify frontend/backend boundaries.
7. Identify test suites.
8. Identify configuration files.
9. Identify database schemas and migrations.
10. Build dependency relationships.
11. Calculate file hashes.
12. Persist discovery results.
13. Generate a project-specific audit plan.

Create a machine-readable project manifest.

Do not assume a project has only one frontend or backend.

Support monorepositories and multiple applications within one repository.

---

8. MASTER AUDIT ORCHESTRATOR

Create a centralized AgentService.

The orchestrator is responsible for:

- Understanding the project manifest.
- Generating the audit plan.
- Selecting specialized agents.
- Assigning tasks.
- Managing dependencies between tasks.
- Scheduling independent tasks concurrently.
- Managing execution state.
- Collecting results.
- Validating evidence.

- Deduplicating findings.
- Handling failures.
- Resuming interrupted audits.
- Generating final reports.

Use a controlled task graph.

Do not allow agents to execute arbitrary shell commands directly.

Agent contract

Every specialized agent must implement a consistent interface:

```
async def execute(
    self,
    context: AuditContext,
    task: AuditTask
) -> AgentResult:
    ...
```

AgentResult must contain:

- Agent name.
- Task ID.
- Execution status.
- Findings.
- Evidence references.
- Metrics.
- Warnings.
- Errors.
- Execution duration.
- Token usage, where applicable.

Validate every result using Pydantic.

Tool registry

Implement a ToolRegistry.

Every tool must define:

- Tool name.
- Description.
- Input schema.
- Output schema.
- Required permissions.
- Execution policy.
- Timeout.
- Whether it is read-only or mutating.

The LLM may request tools only through the registry.

The registry must validate the request before execution.

---

9. SPECIALIZED AGENTS

Build the following specialized agents.

A. Repository Intelligence Agent

Analyze repository structure, technologies, project boundaries, and dependencies.

B. Code Quality Agent

Identify:

- Syntax errors.
- Logical inconsistencies.
- Dead code.
- Duplicate code.
- Poor exception handling.
- Type errors.
- Complexity.
- Unused dependencies.
- Incomplete implementations.

C. Architecture Auditor

Evaluate:

- SOLID principles.
- Separation of concerns.
- Modularity.
- Circular dependencies.
- Coupling.
- Abstraction.
- Dependency injection.
- Scalability.

D. Backend QA Agent

Start the backend and validate:

- Server initialization.
- Routes.
- APIs.
- Request validation.
- Authentication.
- Authorization.
- Database integration.
- Exception handling.
- Background jobs.
- External integrations.

E. Frontend QA Agent

Start the frontend and use Playwright.

Validate:

- Page loading.
- Routes.
- Buttons.
- Forms.
- Navigation.
- State changes.
- API communication.
- Browser console.
- Network failures.
- Responsive behavior.
- UI error states.

F. Database Auditor

Validate:

- Schema.
- Migrations.
- Constraints.
- Relationships.
- CRUD.

- Transactions.
- Persistence.
- Query efficiency.
- Data integrity.

G. Security Auditor

Use deterministic scanners and contextual analysis.

Inspect:

- Secrets.
- Injection vulnerabilities.
- Authentication.
- Authorization.
- XSS.
- CSRF.
- CORS.
- Path traversal.
- File upload handling.
- Vulnerable dependencies.
- Sensitive information exposure.

H. Performance Auditor

Measure:

- API latency.
- Throughput.
- Error rates.
- Database queries.
- CPU.
- Memory.
- Frontend loading.
- Bundle size.
- Concurrent request behavior.

I. Integration Auditor

Test complete user workflows across frontend, backend, database, and external integrations.

J. Remediation Agent

Investigate confirmed findings and produce proposed patches.

The remediation agent must not independently authorize or deploy its changes.

---

10. EXECUTION ENGINE

Implement a common ProjectRunner abstraction.

Create:

- LocalProjectRunner
- DockerProjectRunner

Both must implement the same interface.

Required capabilities:

- Prepare workspace.
- Verify project snapshot.
- Prepare dependencies.
- Execute approved commands.
- Start services.
- Stop services.

- Monitor service health.
- Capture stdout and stderr.
- Capture exit codes.
- Enforce timeouts.
- Collect logs.
- Collect artifacts.
- Clean up.

Local execution

Allow users to select explicitly authorized directories.

Use process isolation and restricted execution policies.

Do not run project code with unrestricted access to the platform's credentials or filesystem.

Docker execution

Use disposable, non-root containers.

Apply:

- CPU limits.
- Memory limits.
- Process limits.
- Disk limits.
- Network restrictions.
- Restricted capabilities.
- Read-only mounts where practical.
- Explicit writable temporary directories.
- Execution timeouts.

Never expose the host Docker socket to audited project containers.

Environment configuration

Do not automatically copy the user's entire `.env` file into the execution environment.

Generate an isolated test configuration.

Use synthetic credentials and disposable test databases.

External network access must be explicitly controlled.

---

11. BACKEND AND FRONTEND VALIDATION

Backend

Discover API routes and execute appropriate test cases.

Validate:

- Success responses.
- Invalid payloads.
- Missing parameters.
- Incorrect types.
- Authentication.
- Authorization.
- Edge cases.
- Unexpected exceptions.
- Persistence.

Frontend

Use Playwright to open the real running application.

Build a browser inspection engine.

Test:

- Application startup.
- Page navigation.
- Forms.
- Buttons.
- Modal interactions.
- Search.
- Filters.
- Pagination.
- Data tables.
- Loading states.
- Error states.
- API communication.
- Responsive layouts.

Collect screenshots, traces, browser console logs, and failed network requests.

Full-stack E2E

Validate complete business workflows.

For example:

User registration → login → dashboard → create record → persist record → retrieve record → update record → delete test record.

Use isolated test data.

Never perform destructive operations against production resources.

---

12. FINDING VALIDATION ENGINE

Create a unified finding model.

Every finding must have:

- Unique ID.
- Category.
- Severity.
- Title.
- Description.
- Project ID.
- Audit session ID.
- File path.
- Line references.
- Evidence.
- Reproduction steps.
- Expected behavior.
- Actual behavior.
- Root cause.
- Impact.
- Recommended fix.
- Confidence.
- Verification status.
- Lifecycle status.
- Fingerprint.

Severity levels:

- Critical
- High
- Medium
- Low
- Informational

Verification states:

- Unverified
- Suspected
- Reproduced
- Confirmed
- Rejected
- Unable to verify

Do not automatically promote a suspected issue to confirmed.

Deduplicate findings across agents using stable fingerprints and evidence.

Retain historical findings across audit sessions.

Distinguish new, recurring, resolved, reopened, and accepted-risk findings.

---

13. EVIDENCE MANAGEMENT

Build a dedicated EvidenceService.

Supported evidence:

- Source references.
- Command output.
- Test output.
- Browser screenshots.
- Browser traces.
- API responses.
- Runtime logs.
- Database verification.
- Scanner reports.
- Performance results.
- Patch diffs.

Every evidence record must have:

- Evidence ID.
- Audit session ID.
- Associated task.
- Associated finding, if applicable.
- Evidence type.
- Storage location.
- Checksum.
- Timestamp.
- Source tool.
- Relevant metadata.

Evidence must be traceable.

Do not expose credentials, secrets, personal data, or sensitive environment values in reports or logs.

---

14. AI PROVIDER GATEWAY

Implement an abstraction:

```
class LLMProvider:
    async def generate(self, request):
        ...

    async def generate_structured(
        self,
        request,
        response_schema
```

```
    ):
        ...
```

Support Gemini, OpenAI, and Ollama.

Requirements:

- Provider-independent interfaces.
- Configurable models.
- Timeout management.
- Retry policies.
- Token usage tracking.
- Cost tracking where available.
- Structured output validation.
- Provider fallback where explicitly configured.
- No API keys in source code.
- No secrets in logs.
- Externalized prompts.
- Context window management.

Use deterministic scanners before invoking LLMs.

Do not send an entire large repository into a single model request.

Use targeted file selection, chunking, relevant code context, and dependency-aware analysis.

Avoid unnecessary LLM calls.

---

15. REMEDIATION AND APPROVAL SYSTEM

The remediation process must use the following sequence:

1. Confirm the finding.
2. Identify the root cause.
3. Identify affected files.
4. Generate a minimal proposed patch.
5. Validate patch syntax.
6. Run static checks on the proposed patch.
7. Generate a complete Git diff.
8. Calculate patch checksum.
9. Present impact analysis.
10. Request explicit human approval.
11. Create a dedicated working branch or isolated working copy.
12. Verify the source snapshot has not unexpectedly changed.
13. Apply the approved patch.
14. Run regression tests.
15. Compare pre-fix and post-fix behavior.
16. Record the result.
17. Mark the issue resolved only if the verification criteria pass.

Approval must be bound to the exact patch checksum.

If the patch changes after approval, request approval again.

Never automatically merge or deploy.

---

16. QUALITY SCORING AND RELEASE GATES

Create separate quality dimensions:

- Code quality.
- Backend reliability.
- Frontend quality.
- Architecture.

- Security.
- Integration.
- Performance.
- Test quality.

Provide transparent scoring criteria.

Never confuse test coverage with product quality.

Release gate states:

- APPROVED
- APPROVED_WITH_WARNINGS
- REJECTED
- INCOMPLETE_AUDIT

Unresolved critical defects must block approval.

Incomplete essential checks must produce INCOMPLETE_AUDIT.

Allow configurable organizational policies.

Keep scoring separate from blocking conditions.

---

17. FRONTEND DASHBOARD

Build a premium, professional engineering dashboard.

Avoid unnecessary decoration.

Use a clear information hierarchy.

Dashboard pages

1. Overview.
2. Projects.
3. Project details.
4. Repository explorer.
5. Audit sessions.
6. Live audit monitoring.
7. Findings.
8. Finding details.
9. Evidence explorer.
10. Test execution results.
11. Security assessment.
12. Performance assessment.
13. Remediation proposals.
14. Approval center.
15. Reports.
16. Agent activity.
17. System settings.

Overview dashboard

Display:

- Registered projects.
- Total audits.
- Active audits.
- Completed audits.
- Failed audits.
- Total findings.
- Critical findings.
- Security findings.
- Pending approvals.

- Average audit duration.

Project details

Display:

- Project architecture.
- Technologies.
- Repository file count.
- Inspection coverage.
- Last audit.
- Audit history.
- Quality metrics.
- Unresolved issues.
- Current execution status.

Live audit monitoring

Show:

- Current phase.
- Current task.
- Active agents.
- Completed tasks.
- Failed tasks.
- Execution logs.
- Progress.
- Elapsed time.
- Cancellation controls.

Progress must reflect actual completed tasks, not an arbitrary animated percentage.

Findings explorer

Support filtering by:

- Severity.
- Category.
- Agent.
- File.
- Status.
- Audit session.
- Verification state.

Approval center

Show:

- Original code.
- Proposed code.
- Full diff.
- Finding description.
- Root cause.
- Potential impact.
- Test plan.
- Approval and rejection actions.

---

18. REPORT GENERATION

Support HTML and PDF reports.

Include:

1. Executive summary.
2. Project overview.

3. Technology stack.
4. Repository inventory.
5. Inspection coverage.
6. Architecture findings.
7. Code quality findings.
8. Backend results.
9. Frontend results.
10. API testing.
11. E2E testing.
12. Database assessment.
13. Security findings.
14. Performance findings.
15. Dependency findings.
16. Test results.
17. Incomplete checks.
18. Proposed remediations.
19. Applied fixes.
20. Regression results.
21. Release recommendation.

Every result must link to its supporting evidence.

Never claim an audit is complete when essential tasks have not run.

---

19. PROJECT-INTEGRATED CLI

Create a distributable Python package called qc-agent.

Support:

```
qc-agent discover

qc-agent audit --full

qc-agent audit --code

qc-agent audit --backend

qc-agent audit --frontend

qc-agent audit --security

qc-agent audit --e2e

qc-agent audit --full --runner docker

qc-agent findings

qc-agent report --format html

qc-agent repair --propose

qc-agent repair --apply PROPOSAL_ID

qc-agent status

qc-agent config validate
```

Create a project-level `.qc/config.yaml`.

Support:

- Project exclusions.
- Custom audit rules.
- Execution mode.
- Test configuration.
- Service startup configuration.
- Test credentials references.
- Quality gate policies.
- Reporting configuration.

The CLI must be capable of running a local audit without the central dashboard.

When connected, it should synchronize audit sessions and findings with the central platform.

Never transmit source code or evidence to a central service unless explicitly configured and authorized.

---

20. REST API DESIGN

Implement versioned APIs.

Suggested endpoints:

Projects:

```
POST   /api/v1/projects
GET    /api/v1/projects
GET    /api/v1/projects/{project_id}
DELETE /api/v1/projects/{project_id}
```

Discovery:

```
POST   /api/v1/projects/{project_id}/discover
GET    /api/v1/projects/{project_id}/manifest
```

Audits:

```
POST   /api/v1/projects/{project_id}/audits
GET    /api/v1/audits
GET    /api/v1/audits/{audit_id}
POST   /api/v1/audits/{audit_id}/cancel
GET    /api/v1/audits/{audit_id}/progress
```

Findings:

```
GET    /api/v1/audits/{audit_id}/findings
GET    /api/v1/findings/{finding_id}
PATCH  /api/v1/findings/{finding_id}
```

Evidence:

```
GET    /api/v1/findings/{finding_id}/evidence
```

Remediation:

```
POST   /api/v1/findings/{finding_id}/proposals
GET    /api/v1/remediation/{proposal_id}
POST   /api/v1/remediation/{proposal_id}/approve
POST   /api/v1/remediation/{proposal_id}/reject
POST   /api/v1/remediation/{proposal_id}/apply
```

Reports:

```
POST   /api/v1/audits/{audit_id}/reports
GET    /api/v1/audits/{audit_id}/reports
```

Agent and system status:

```
GET    /api/v1/system/health
GET    /api/v1/system/agents
GET    /api/v1/system/executors
```

Use authentication, authorization, input validation, pagination, rate limiting, and audit logging.

Use WebSockets or Server-Sent Events for live progress updates.

---

21. IMPLEMENTATION PHASES

Develop the product in the following sequence.

PHASE 0 — Architecture and foundation.

PHASE 1 — Project registration and repository discovery.

PHASE 2 — Static code analysis and finding management.

PHASE 3 — Local and Docker execution engines.

PHASE 4 — Backend and API testing.

PHASE 5 — Frontend browser testing.

PHASE 6 — Database, integration, security, and performance testing.

PHASE 7 — Master orchestrator and specialized agents.

PHASE 8 — AI-powered remediation and approval workflows.

PHASE 9 — CLI integration, reporting, CI/CD, and production hardening.

Do not generate the entire system in one unverified pass.

Implement each phase as a working vertical slice.

Every phase must include:

1. Architecture updates.
2. Backend implementation.
3. Database changes.
4. Frontend integration where relevant.
5. Unit tests.
6. Integration tests.
7. Actual execution.
8. Error diagnosis.
9. Fixes.
10. Acceptance testing.
11. Documentation.

Maintain a development progress document and a technical decision log.

---

22. REQUIRED DEVELOPMENT WORKFLOW

Follow this workflow continuously.

STEP 1 — Inspect the current repository.

STEP 2 — Understand existing architecture.

STEP 3 — Create the implementation plan.

STEP 4 — Implement the smallest complete vertical slice.

STEP 5 — Run the backend.

STEP 6 — Run the frontend.

STEP 7 — Run relevant tests.

STEP 8 — Inspect runtime logs.

STEP 9 — Verify database persistence.

STEP 10 — Validate API and frontend integration.

STEP 11 — Identify the root cause of failures.

STEP 12 — Apply the smallest safe correction.

STEP 13 — Repeat validation.

STEP 14 — Update implementation documentation.

Do not mark a phase complete based on static code inspection.

A phase is complete only after its acceptance tests have passed.

Do not redesign or remove existing functionality unnecessarily.

Do not replace working components merely because another technology is available.

Preserve existing decisions unless evidence demonstrates a technical problem.

---

23. SELF-AUDIT REQUIREMENT

Once the platform is operational, audit the platform itself.

The platform must:

- Discover its own repository.
- Inspect its backend.
- Inspect its frontend.
- Start its backend.
- Start its frontend.
- Test its APIs.
- Execute browser workflows.
- Validate database persistence.
- Execute security checks.
- Generate a complete audit report.

Introduce a deliberately vulnerable or defective fixture project in a controlled test environment.

Verify the entire defect lifecycle:

Detection → Evidence → Confirmation → Proposed patch → Human approval → Application → Regression tests → Resolution.

Never introduce test vulnerabilities into the production application itself.

---

24. FINAL DEFINITION OF DONE

The product is not complete until the following capabilities are demonstrably operational:

- Standalone project registration.
- Project-integrated CLI.
- Local project execution.
- Docker project execution.
- Complete repository inventory.
- File inspection tracking.
- Multi-language discovery.

- Static code analysis.
- Backend startup and API testing.
- Frontend startup and browser automation.
- Database validation.
- E2E workflows.
- Security scanning.
- Performance testing.
- AI agent orchestration.
- Evidence collection.
- Finding management.
- Quality scoring.
- HTML and PDF reporting.
- Proposed code fixes.
- Human approval workflow.
- Patch application.
- Regression verification.
- Audit history.
- Interrupted audit recovery.
- Project access control.
- Execution isolation.
- Self-audit capability.

For every feature, demonstrate actual execution.

If a feature is incomplete, clearly label it as incomplete.

Never create placeholder functionality and describe it as production-ready.

---

25. INITIAL EXECUTION INSTRUCTION

Before writing application code:

1. Analyze the requirements in this specification.
2. Produce the detailed system architecture.
3. Produce the database ER diagram.
4. Produce the module dependency diagram.
5. Define agent interfaces and tool contracts.
6. Define the execution isolation model.
7. Define API contracts.
8. Define the audit state machine.
9. Define the remediation approval state machine.
10. Define the Phase 0 implementation checklist.

Then begin implementation.

Do not stop after producing the architecture document.

Implement Phase 0 and validate it.

After Phase 0 is completed, continue to Phase 1 and subsequent phases in sequence, keeping the project executable at every milestone.

Maintain truthful progress tracking.

If blocked, identify the exact technical blocker and its evidence instead of pretending the implementation is complete.

Your ultimate objective is to deliver a reusable, production-grade Autonomous QC & Project Audit Intelligence Platform that can independently assess different software projects and provide verifiable evidence of their technical quality.

---

14. How I recommend executing this in Antigravity

Do not ask Antigravity to build all ten phases in one attempt. That will likely produce a large amount of code with incomplete integrations.

Use this execution strategy:

Step 1 — Paste the master prompt.

Ask it to analyze the requirements and create the architecture, ER diagram, module structure, and Phase 0 implementation plan.

Step 2 — Start implementation.

Give it this instruction:

Writing

Start implementing Phase 0 of the QC Intelligence Platform.

First inspect the current workspace and verify the architecture against the master specification.

Implement the actual backend, frontend, PostgreSQL integration, authentication foundation, project registration, audit session management, structured logging, and database migrations.

Do not generate placeholder APIs or static dashboard components.

Run the backend and frontend, execute tests, verify database persistence, and validate their integration.

Fix all reproducible implementation defects before marking Phase 0 complete.

Produce a detailed Phase 0 completion report containing implemented modules, files created, commands executed, actual test results, remaining issues, and evidence.

Do not start Phase 1 until Phase 0 passes its acceptance criteria.

Step 3 — Continue phase by phase.

After each completed phase, instruct Antigravity to proceed to the next phase, preserving existing architecture and functionality.

Step 4 — Perform complete self-audit.

Once all phases are implemented, run the QC agent against its own repository.

15. Final architectural decisions

These are the decisions I would lock before starting development:

Decision	Selection
Product	Standalone + integrated CLI
Backend	FastAPI
Frontend	React + Vite
Database	PostgreSQL
Agent architecture	Custom orchestrator + specialized agents
LLM	Gemini + OpenAI + Ollama
Execution	Local + Docker
Browser testing	Playwright
API testing	Pytest + HTTPX
Security	Semgrep + Bandit + OSV
Remediation	Human approval required
Reporting	HTML + PDF
Architecture	Modular monorepo
Development	Vertical slices

The most important product principle

The agent should be able to answer three questions for every project:

1. What did I inspect?

Exact file coverage, modules, endpoints, and workflows.

2. What did I actually test?

Real execution results, logs, screenshots, database checks, and security scan results.

3. What remains unverified or broken?

Confirmed defects, potential risks, incomplete checks, and limitations.

If it cannot answer these three questions accurately, it is not a reliable QC agent.

My strongest recommendation is to prioritize the **execution engine, repository coverage tracker, and evidence management** before sophisticated AI agents. Those three components determine whether the eventual product is a genuine engineering quality-control system or just another AI tool that reads code and generates convincing reports.

Architecture and workflow recommendations incorporate your existing FastAPI, React/Vite, PostgreSQL/SQLAlchemy, multi-provider AI, and autonomous verification experience.